

Infosys Springboard Virtual Internship 6.0 Completion Report

Team Details <Do not mention any personally identifiable information like email ID, institute details, mobile phone number etc.>

Batch Number:13

Start date:02-02-2026

Names:

1. Amatullah
2. Tanvi Nema
3. Manchuri Praneeth Reddy
4. Yaswanth Kumar Kanamarlapudi
5. M Mani Krishna
6. Kachhatiya Jaydip Vallabhbhai
7. Sravyasree Gedela
8. Nandana Biju
9. Hariprasad R
10. Dasari Bhuvan
11. Morasa Guru Sandeep
12. Vasudev V
13. Haristha S
14. Krishegowda
15. Pothala Jayanth
16. Manas Gahoi
17. Upasana Sandeep Wagh
18. Gajanan Babarao Jagdambe
19. Rithanya S

Internship Duration: 8 Weeks

1. Project Title

Intelligent Poaching Detection and Response System

2. Project Objective

The primary goal of the internship project, ***“GuardianAI: Intelligent Poaching Detection and Response System,”*** was to design and implement an AI-powered solution capable of accurately detecting poachers, weapons, and wildlife from surveillance images and video footage. The project focused on creating an automated monitoring system for forested areas that could identify potential threats in real time and generate timely alerts, enabling rapid response and effective intervention. This initiative not only aimed to enhance conservation efforts but also demonstrated the practical application of AI and computer vision techniques in solving real-world environmental challenges.

This solution plays a critical role in supporting wildlife conservation and forest security initiatives. By providing real-time detection and automated alert notifications, it enables authorities to respond swiftly

to illegal activities such as poaching. Additionally, the project demonstrates the practical application of deep learning and modern web technologies in addressing real-world environmental and security challenges, making it a scalable and efficient tool for effective forest monitoring and protection

3. Project description in detail

The internship project focuses on developing **GuardianAI — an Intelligent Poaching Detection and Response System**, an AI-powered wildlife surveillance platform designed to detect poachers and protect animals using advanced deep learning techniques.

Approach:

The project follows a structured approach starting with dataset preparation and training a **YOLOv8 object detection model** to identify four key classes: poacher, weapon, animal, and ranger. The model processes both images and video frames to perform real-time detection. A complete system architecture was designed consisting of a frontend, backend, database, and AI model integration. The backend was developed using **FastAPI**, which handles image uploads, triggers model inference, and manages alerts. The YOLOv8 model performs detection, and based on the results, the system generates alerts for critical threats such as poachers or weapons.

The frontend was developed using **React** and provides role-based dashboards for rangers and officers. Rangers can upload surveillance images and view detection results, while officers can monitor alerts and take action through a dedicated dashboard.

Additionally, an automated **email alert system** was integrated, which sends notifications with detection details and annotated images whenever a high-risk threat is detected.

Approach:

Built an end-to-end pipeline: upload → AI detection → results → alerts → database logging. Implemented automated email alerts with detected images and location. Designed a responsive command-center UI for monitoring and analytics.

Technologies Used:

- Python
- YOLOv8 (Ultralytics) for object detection
- FastAPI for backend API development
- MongoDB for database management
- React (with Vite and TailwindCSS) for frontend development
- OpenCV for image and video processing
- JWT Authentication for role-based access
- SMTP (Gmail) for email alert system

Real-World Impact:

This project has significant real-world applications in wildlife conservation and forest security. It enables automated monitoring of large forest areas, reducing the need for constant human surveillance.

By detecting poachers and weapons in real time and sending instant alerts to authorities, the system helps prevent illegal hunting and protect endangered species.

The role-based dashboard system improves coordination between field rangers and officers, ensuring faster response to threats. Overall, the project demonstrates how AI and modern web technologies can be integrated to build a scalable and impactful solution for environmental protection and security.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Data collection, initial planning	Requirement analysis and scope finalisation
Week 2	Labeling and preprocessing	Dataset preparation and augmentation
Week 3	Model development	YOLOv8 model setup and training
Week 4	Model evaluation	Performance testing and tuning
Week 5	Backend integration	AI API + database logging
Week 6	Frontend development	Upload, detection UI, dashboards
Week 7	Alert system + polish	Email alerts, authentication, UI fixes
Week 8	Review & documentation	Final testing, report, presentation

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Requirements finalized and scope defined	02-02-2026
Prototype/First Draft	Basic detection flow and UI ready	15-02-2026
Mid-Term Review	End-to-end AI + backend integration	01-03-2026
Final Submission	Full system completed with alerts	15-03-2026
Presentation	Demo and documentation prepared	22-03-2026

5b. Project execution details

The project was executed in multiple phases, following a structured approach to design and develop an intelligent poaching detection system.

- Problem Understanding & Planning:

The project started with identifying the need for an automated wildlife monitoring system to detect poachers and protect animals. Key requirements such as real-time detection, alert generation, and role-based access were defined.

- Data Collection & Preparation:

Datasets were collected from multiple sources containing images of animals, humans (poachers), and weapons. The data was organized into defined classes and preprocessed using techniques such as resizing, normalization, and formatting suitable for the YOLOv8 model.

- Model Development:

A **YOLOv8 object detection model** was used to detect four classes: poacher, weapon, animal, and ranger. The model was trained on the prepared dataset and evaluated using performance metrics such as **accuracy, precision, recall, and F1-score**.

The model achieved an approximate **accuracy of 85–90%**, with **precision in the range of 0.7–0.8** for critical detections like poachers and weapons. The **F1-score** indicated a balanced performance, ensuring reliable and consistent predictions. Further improvements were considered through hyperparameter tuning and dataset optimisation.

- **Backend Development:**

The backend was developed using **FastAPI**, which handled API creation, user authentication, and communication between components. It managed image uploads, triggered the model for detection, and stored results in a **MongoDB database**.

- **Frontend Development:**

A user-friendly frontend was developed using **React** and styled with TailwindCSS. Role-based dashboards were created for rangers and officers. Rangers could upload images and view detection results, while officers could monitor alerts and manage threats.

- **Alert & Notification System:**

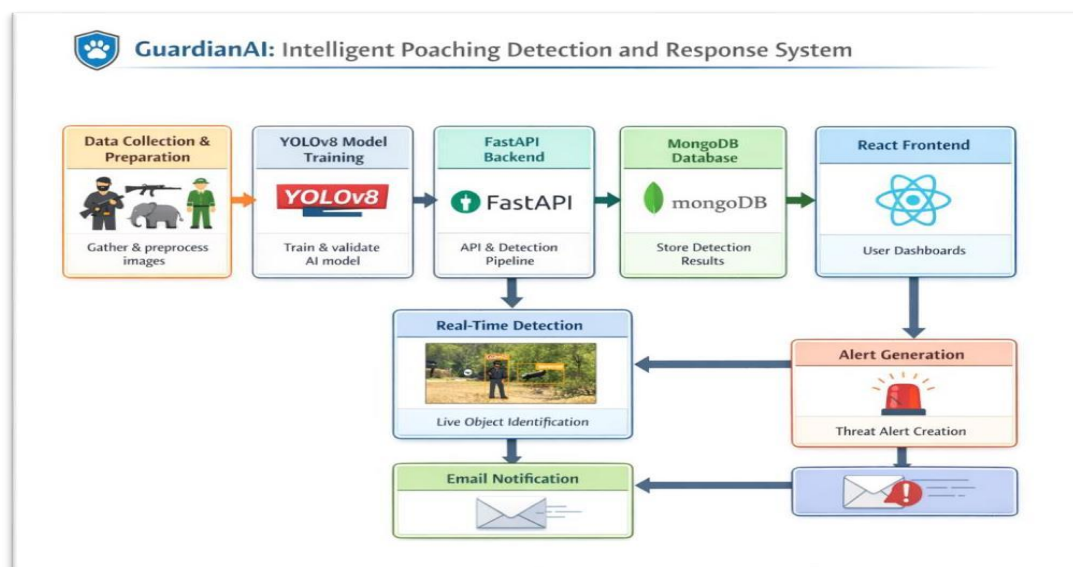
An automated email alert system was implemented using SMTP services. When a poacher or weapon was detected, the system generated alerts and sent email notifications with detection details and annotated images.

- **System Integration & Testing:**

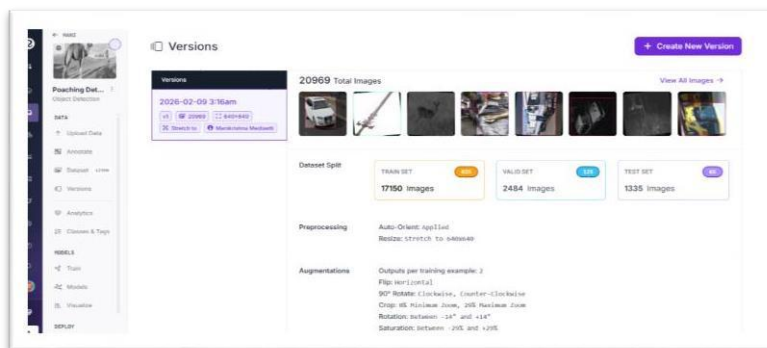
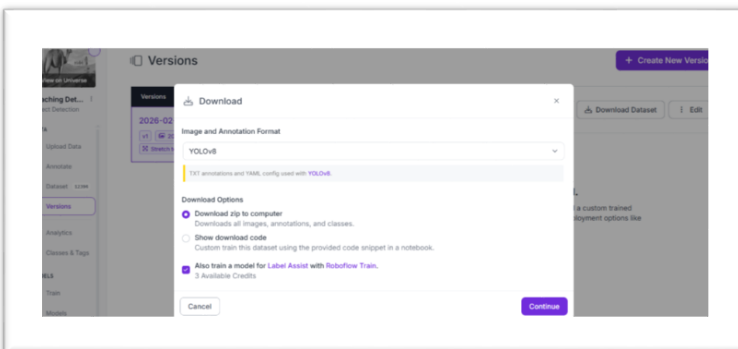
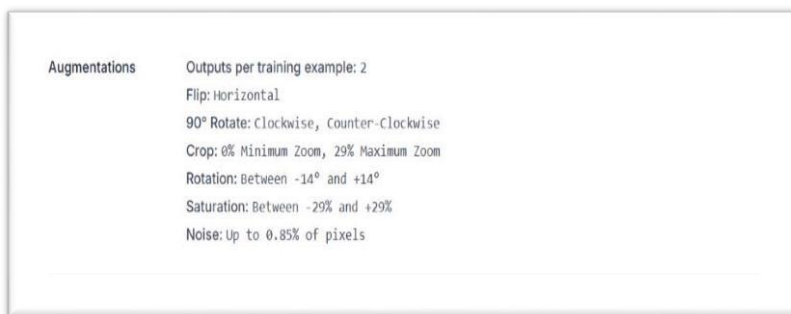
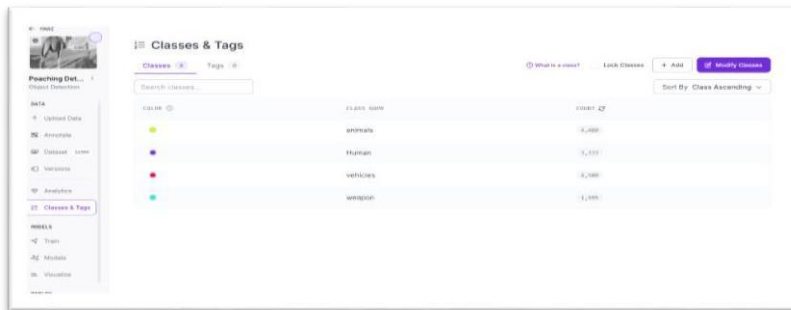
All components, including the model, backend, frontend, and database, were integrated into a complete system. The application was tested using various inputs to ensure accurate detection and smooth workflow.

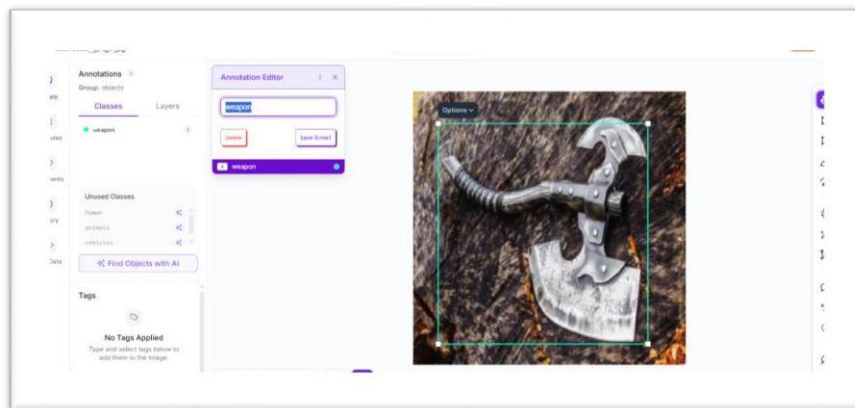
- **Deployment & Finalization:**

The system was configured to run locally with both frontend and backend servers. Final testing was conducted to ensure the system met all project requirements and was ready for demonstration.



6. Snapshots / Screenshots





Preprocessing

Auto-Orient: Applied

Resize: Stretch to 640x640

20969 Total Images

[View All Images →](#)



Dataset Split

TRAIN SET

82%

17150 Images

VALID SET

12%

2484 Images

TEST SET

6%

1335 Images

```

Downloading https://github.com/ultralytics/assets/releases/download/v8.0.0/yolo26n.pt to 'yolo26n.pt': 100% — 5.3PB 115.0MB/s 0.8s
Ultralytics 8.4.14 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=RandAugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% — 755.1KB 24.8MB/s 0.0s
Overriding model.yaml nc=80 with nc=5

      from  n  params  module  arguments
0         -1  1      464  ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1         -1  1     4672  ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2         -1  1     7360  ultralytics.nn.modules.block.C2f   [32, 32, 1, True]
3         -1  1    18560  ultralytics.nn.modules.conv.Conv  [32, 64, 3, 2]
4         -1  1    48664  ultralytics.nn.modules.block.C2f   [64, 64, 2, True]
5         -1  1    73984  ultralytics.nn.modules.conv.Conv  [64, 128, 3, 2]
6         -1  2   197632  ultralytics.nn.modules.block.C2f   [128, 128, 2, True]
7         -1  1   295424  ultralytics.nn.modules.conv.Conv  [128, 256, 3, 2]
8         -1  1   460288  ultralytics.nn.modules.block.C2f   [256, 256, 1, True]
9         -1  1   164608  ultralytics.nn.modules.block.SPPF  [256, 256, 1]
10        -1  1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
11       [-1, 6] 1         0  ultralytics.nn.modules.conv.Concat  [1]
12        -1  1   148224  ultralytics.nn.modules.block.C2f   [384, 128, 1]
13        -1  1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
14       [-1, 4] 1         0  ultralytics.nn.modules.conv.Concat  [1]
15        -1  1   37248  ultralytics.nn.modules.block.C2f   [192, 64, 1]
16        -1  1   36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
17       [-1, 12] 1         0  ultralytics.nn.modules.conv.Concat  [1]
18        -1  1   123648  ultralytics.nn.modules.block.C2f   [192, 128, 1]
19        -1  1   147712  ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
20       [-1, 9] 1         0  ultralytics.nn.modules.conv.Concat  [1]
21        -1  1   493056  ultralytics.nn.modules.block.C2f   [384, 256, 1]
22      [15, 18, 21] 1   752287  ultralytics.nn.modules.head.Detect  [5, 16, None, [64, 128, 256]]

Model summary: 130 layers, 3,011,823 parameters, 3,011,807 gradients, 8.2 GFLOPs

```

```

Copy of Poaching_Detection_YOLOv8.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all +

Download https://github.com/ultralytics/assets/releases/download/v8.0.0/yolo26n.pt to 'yolo26n.pt': 100% — 5.3PB 115.0MB/s 0.8s
AMP: checks passed
train: Fast image access (ping: 0.010.0 ms, read: 1057.31477.6 MB/s, size: 51.0 KB)
train: Scanning /content/KG-Intern-1/train/labels... 14242 images, 164 backgrounds, 0 corrupt: 100% — 14242/14242 2.3k/s 6.2s
train: New cache created: /content/KG-Intern-1/train/labels.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_val: Fast image access (ping: 0.010.0 ms, read: 766.51541.5 MB/s, size: 49.5 KB)
val: Scanning /content/KG-Intern-1/valid/labels... 1649 images, 18 backgrounds, 0 corrupt: 100% — 1649/1649 1.0k/s 1.6s
val: New cache created: /content/KG-Intern-1/valid/labels.cache
optimizer: 'optimizer-auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr' and 'momentum' automatically...
optimizer: AdamW(lr=0.01, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)
Plotting labels to /content/runs/detect/train/labels.jpg...
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/detect/train
Starting training for 50 epochs...

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/50    2.8G    1.477    2.497    1.518      7          640: 100% — 891/891 2.6it/s 5:44
      Class  Images  Instances  Box(P  mAP50-95): 100% — 52/52 3.6it/s 14.5s
      all    1649    2955      0.63   0.295   0.335   0.19

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
2/50    2.8G    1.505    1.968    1.54       47          640: 73% — 650/891 5.1it/s 3:40x47.5s

```



```

from ultralytics import YOLO
from google.colab import drive
import torch

# 1. Mount Google Drive (Required to access your dataset)
drive.mount('/content/drive')

# 2. Verify GPU is active
print(f"GPU Available: {torch.cuda.is_available()}")
if not torch.cuda.is_available():
    print("⚠ WARNING: GPU not found! Check Runtime > Change Runtime Type > GPU")

# 3. Load Model
model = YOLO("yolov8n.pt") # 'n' is Nano (fastest for Colab)

# 4. Train
results = model.train(
    data="/content/drive/MyDrive/poacher_detection_dataset/My_project_1.vli.yolov8/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    device=0, # [x] Force GPU usage
    workers=2, # [x] Prevent Colab data loader crashes
    project="/content/drive/MyDrive/poacher_detection_dataset/results", # [x] Save weights to Drive permanently
    name='poacher_exp_1', # [x] Name this specific run
    patience=10, # [x] Stop early if no improvement
    save=True # [x] Ensure checkpoints are saved
)

```

[Intelligent Detection Online](#)

[Go to Dashboard](#)

Automated Poaching Detection System

Protecting wildlife through advanced YOLOv8 surveillance. Upload field camera photos for real-time analysis, alerting authorities before it's too late.

[Upload Surveillance Image →](#)
[Open Dashboard](#)

Real-time YOLOv8 Analysis

Instantly detect poachers, weapons, and vehicles with high accuracy using deep learning.

Automated Alerts

Immediate email notifications to nearest forest stations when suspicious activity is flagged.

Wildlife Monitoring

Track and catalog elephant and tiger movements to better understand migration patterns.

Intelligent Poaching Detection & Response

Advanced computer vision system designed to secure protected terrains, identify threats, and trigger real-time mobilization.

DETECTION
YOLOv8 Powered

RESPONSE
< 2s Latency

Welcome Back

Enter your credentials to access the secure terminal

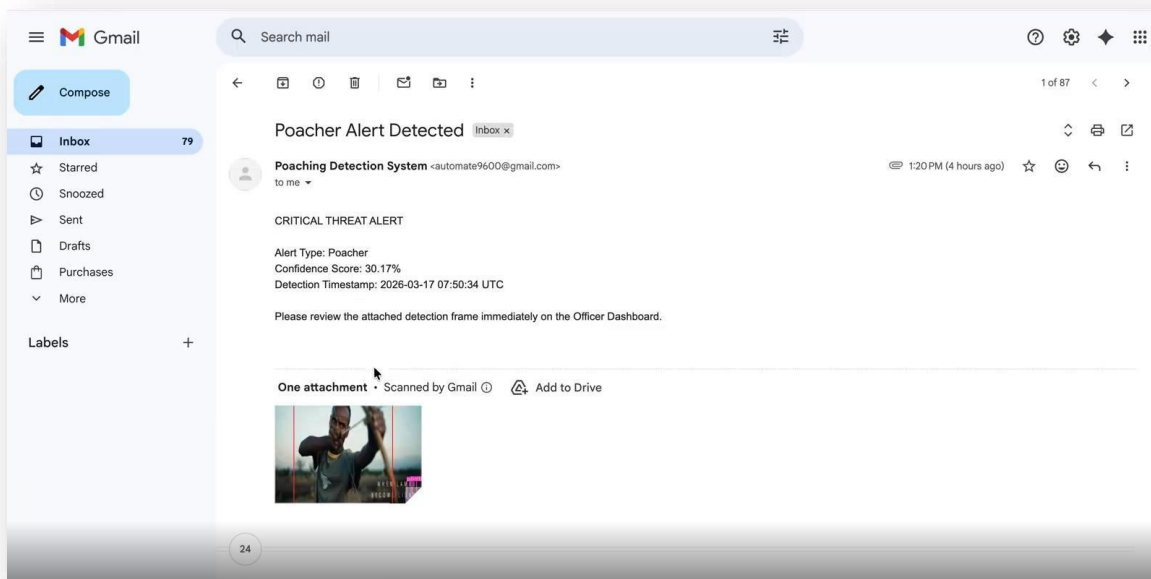
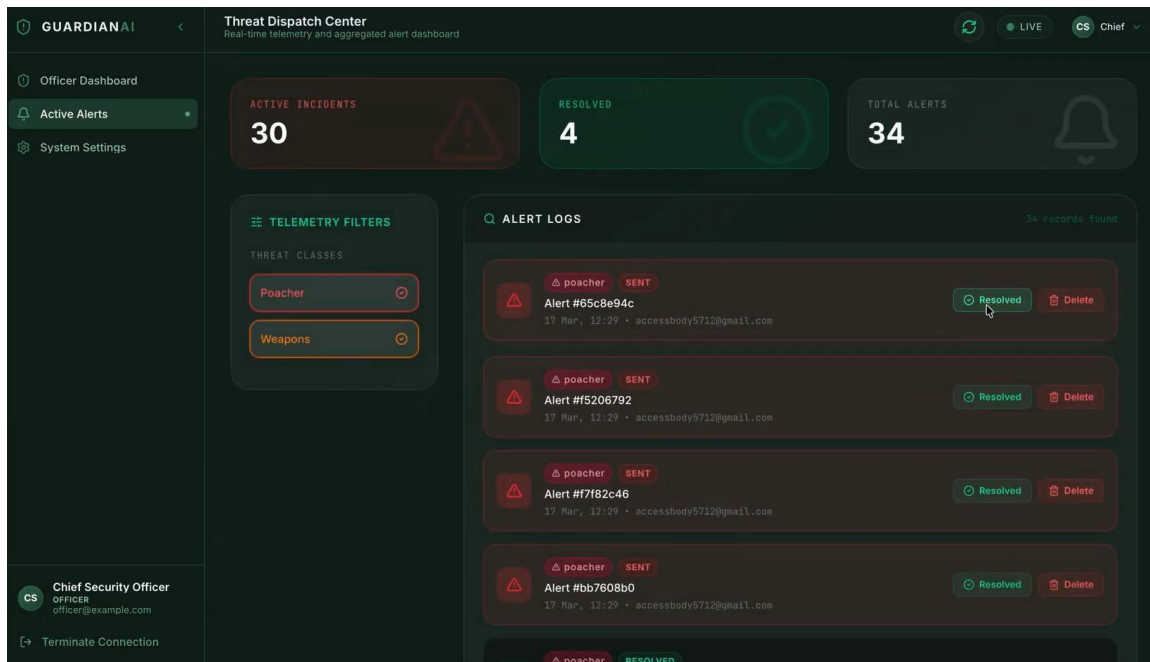
Secure Email ID

Access Key

[Forgot access key?](#)

[Initiate Uplink](#)

[New field agent? Request clearance](#)



7. Challenges Faced

During the internship, several technical, operational, and communication challenges were encountered while developing the poacher detection system.

Dataset Quality and Imbalance:

One of the major challenges was collecting a well-balanced dataset with sufficient images for all classes, especially for poachers and weapons. Some classes had limited data, which affected model performance.

Solution: Data augmentation techniques such as rotation, flipping, and scaling were applied to increase dataset diversity. Additional data sources were also explored to improve balance.

Model Accuracy and False Detections:

The model initially produced false positives, particularly confusing humans with poachers or misclassifying objects.

Solution: The model was fine-tuned by adjusting hyperparameters and improving training data quality. Performance metrics such as precision and recall were monitored to achieve better results.

Integration of Model with Backend:

Integrating the YOLOv8 model with the FastAPI backend was challenging, especially handling image processing and asynchronous tasks.

Solution: The detection pipeline was modularized, and background task handling was implemented to ensure smooth processing without blocking API responses.

Handling Large Image/Video Data:

Processing multiple images and video frames required efficient handling to avoid performance issues.

Solution: OpenCV was used for optimized image processing, and batch handling techniques were applied to manage multiple inputs efficiently.

Frontend–Backend Communication:

Ensuring smooth communication between the React frontend and FastAPI backend, including authentication and API integration, was challenging.

Solution: Axios was used for API calls, and proper endpoint structuring along with JWT-based authentication ensured secure and efficient communication.

Email Alert Configuration:

Setting up automated email notifications using SMTP initially faced issues such as authentication errors and configuration setup.

Solution: Gmail App Passwords were configured, and proper environment variables were used to securely manage credentials.

Team Communication and Coordination:

At times, there were delays due to misunderstandings or lack of clarity in task distribution within the team.

Solution: Regular discussions and updates were conducted to clarify doubts, align tasks, and ensure smooth collaboration among team members.

8. Learnings & Skills Acquired

During this internship, we all gained both technical expertise and practical experience in building a real-world AI-based system.

Technical Skills:

We developed a strong understanding of deep learning and object detection using **YOLOv8**. We also gained hands-on experience in backend development using **FastAPI**, frontend development using **React**, and database management with **MongoDB**. Additionally, we worked with tools like OpenCV for image processing and implemented API integration between frontend and backend systems.

System Design & Integration:

We learned how to design and integrate a complete end-to-end system, combining the AI model, backend, frontend, and database. This helped us understand real-world application architecture beyond just model building.

Model Evaluation & Optimization:

We all gained knowledge of evaluating models using metrics such as accuracy, precision, recall, and F1-score. We also understood the importance of data quality, preprocessing, and fine-tuning to improve model performance.

Domain Knowledge:

The project provided insights into wildlife conservation and the challenges involved in detecting poaching activities. We understood how AI can be applied to solve real-world environmental and security problems.

Problem-Solving Skills:

We improved my ability to troubleshoot issues related to model performance, system integration, and data handling through logical, analytical thinking.

Communication & Teamwork:

We enhanced my communication skills through regular discussions, presentations, and collaboration with team members. This helped improve coordination and ensure effective project execution.

9. Testimonials from the team

During the internship, we had a valuable learning experience working on a real-world project focused on wildlife protection using AI.

One of the key success points was successfully developing a system that can detect poachers, weapons, and animals from images and video inputs using a YOLOv8 model.

We were able to contribute to building an end-to-end solution by integrating the deep learning model with a backend using FastAPI and designing a frontend interface for user interaction.

Successfully implementing features like real-time detection, alert generation, and email notifications was a major achievement.

Another important success point was achieving good model performance with reliable accuracy and precision, making the system practical for real-world usage.

Additionally, collaborating with the team, resolving technical challenges, and completing assigned tasks on time helped improve my confidence and problem-solving skills.

Overall, this internship provided hands-on experience in applying AI to solve real-world problems and strengthened both my technical and professional skills.

10. Conclusion

The internship was a highly enriching experience that provided practical exposure to building a real-world AI-based system. Working on the **GuardianAI: Intelligent Poaching Detection and Response System** helped us understand how theoretical concepts in data science and machine learning can be applied to solve real-world problems.

The project had a meaningful impact as it focused on wildlife conservation by enabling automated detection of poachers and generating timely alerts. This experience gave us insights into designing scalable systems, integrating AI models with web technologies, and handling real-time data processing.

This internship strongly aligns with our academic background in data science and my career goal of becoming a machine learning engineer. It enhanced our technical skills in deep learning, system development, and problem-solving, while also improving our teamwork and communication abilities. Overall, it has motivated us further to explore AI-driven solutions for impactful real-world applications.

11. Acknowledgements

We sincerely thank the organisation for allowing us to work on such a meaningful and impactful project, which has significantly strengthened our technical expertise, leadership, and problem-solving skills.

We all would like to especially thank our mentor, **Disha Ma'am**, for her continuous guidance, support, and valuable insights throughout the internship. Her mentorship helped us understand complex concepts and stay motivated while working on the project.

We also extend our gratitude to **K Anantha Laxmi** mam for her support and assistance during the internship. Her guidance and encouragement played an important role in completing the assigned tasks.

Finally, we sincerely appreciate all our team members for their dedication, cooperation, and collaborative spirit, which played a key role in making this journey successful, productive, and enjoyable.

